

Exemple de soumission d'activité

ÉTS - LOG430 - Architecture logicielle - Été 2026

Étudiant(e) : Chris-Emmanuel Berton

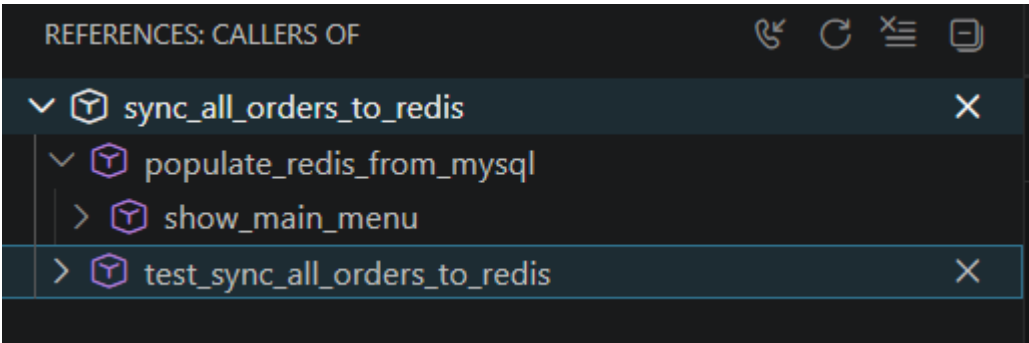
Questions

(Il est obligatoire d'ajouter du code, des captures d'écran ou des sorties de terminal pour illustrer chacune de vos réponses.)

Question 1 : Lorsque l'application démarre, la synchronisation entre Redis et MySQL est-elle initialement déclenchée par quelle méthode ? Veuillez inclure le code pour illustrer votre réponse.

Réponse Qui appelle sync La synchronisation Redis-MySQL se fait par la fonction `populate_redis_from_mysql()` puisqu'elle est la première fonction à appeler `sync_all_orders_to_redis`

```
✓ def populate_redis_from_mysql():  
    """ Populate Redis with orders from MySQL, only if Redis is empty """  
    sync_all_orders_to_redis()
```



```
REFERENCES: CALLERS OF  
✓ sync_all_orders_to_redis  
  ✓ populate_redis_from_mysql  
    > show_main_menu  
      > test_sync_all_orders_to_redis
```

Question 2 : Quelles méthodes avez-vous utilisées pour lire des données à partir de Redis ? Veuillez inclure le code pour illustrer votre réponse.

La lecture des données par le Redis se fait à partir de la fonction `get_orders_from_redis`

```
def get_orders_from_redis(limit=9999):  
    """Get last X orders"""  
    r = get_redis_conn()  
    order_ids = r.lrange("orders", 0, limit - 1)  
    print(limit)  
  
    orders = []  
    order_ids = r.zrevrange("orders", 0, limit - 1)  
  
    return [  
        r.hgetall(f"order:{order_id}")  
        for order_id in order_ids  
    ]
```

Question 3 : Quelles méthodes avez-vous utilisées pour ajouter des données dans Redis ? Veuillez inclure le code pour illustrer votre réponse.

L'ajout des données se fait par la fonction `add_order_to_redis`. Ladite fonction se fait appeler par `add_order`

```
def add_order_to_redis(order_id, user_id, total_amount, items):  
    """Insert order to Redis"""  
  
    r = get_redis_conn()  
    print(r)  
    """  
    Store order and order items in Redis  
    """  
  
    # Store main order  
    r.hset(  
        f"order:{order_id}",  
        mapping={  
            "id": order_id,  
            "user_id": user_id,  
            "total_amount": total_amount  
        }  
    )  
  
    # Store order items  
    for index, item in enumerate(items):  
  
        r.hset(  
            f"order:{order_id}:item:{index}",  
            mapping={  
                "product_id": item["product_id"],  
                "quantity": item["quantity"],  
                "unit_price": item["unit_price"]  
            }  
        )  
  
    # Keep track of newest orders  
    r.lpush("orders", order_id)
```

Question 4 : Quelles méthodes avez-vous utilisées pour supprimer des données dans Redis ? Veuillez inclure le code pour illustrer votre réponse.

La suppression des données se fait par la méthode `delete_order_from_redis`, la méthode étant appelée par `delete_order`

```
def delete_order(order_id: int):
    """Delete order in MySQL, keep Redis in sync"""
    session = get_sqlalchemy_session()
    try:
        order = session.query(Order).filter(Order.id == order_id).first()

        if order:
            session.delete(order)
            session.commit()

            # TODO: supprimer la commande à Redis
            delete_order_from_redis(order_id)
            return int(1)
        else:
            return int(0)

    except Exception as e:
        session.rollback()
        raise e
    finally:
        session.close()
```

```
def delete_order_from_redis(order_id):
    """Delete order from Redis"""
    # Delete main order hash

    r = get_redis_conn()
    print(r)

    r.delete(f"order:{order_id}")

    # Find all item keys
    item_keys = r.keys(f"order:{order_id}:item:*")

    # Delete all item hashes
    if item_keys:
        r.delete(*item_keys)

    # Remove order ID from latest orders list
    r.lrem("orders", 0, str(order_id))
```

Question 5 : Si nous souhaitions créer un rapport similaire, mais présentant les produits les plus vendus, les informations dont nous disposons actuellement dans Redis sont-elles suffisantes, ou devrions-nous chercher dans les tables sur MySQL ? Si nécessaire, quelles

informations devrions-nous ajouter à Redis ? Veuillez inclure le code pour illustrer votre réponse.

Oui : les informations sur la commande permettent de déduire autant les plus grand acheteurs en liant les id au total dépensé en commande que les articles les plus vendus en liant les id des produits à leurs quantité. Dans les deux scénarios, l'information interprétée reste le OrderItem.

```
class OrderItem(Base):
    __tablename__ = 'order_items'

    id = Column(Integer, primary_key=True, autoincrement=True)
    order_id = Column(Integer, ForeignKey('orders.id'), nullable=False)
    product_id = Column(Integer, nullable=False)
    quantity = Column(Integer, nullable=False)
    unit_price = Column(Float, nullable=False)

    # Relationship back to order
    order = relationship("Order", back_populates="order_items")
```

```
def get_highest_spending_users():
    """Get report of highest spending users"""
    r = get_redis_conn()
    keys = r.keys("order:*")
    orders = []

    for key in keys:
        if ":item:" in key:
            continue
        data = r.hgetall(key)
        if not data:
            continue
        orders.append(data)

    expenses_by_user = defaultdict(float)

    for order in orders:
        expenses_by_user[int(order['user_id'])] += float(order['total_amount'])

    highest_spending_users = sorted(
        expenses_by_user.items(),
        key=lambda item: item[1],
        reverse=True
    )
    result = []

    for user_id, total in highest_spending_users[:10]:
        result.append(
            f"<li>{user_id}: {total:.2f}$</li>"
        )

    return "".join(result)
```

```
def get_best_sellers():
    """Get report of best selling products"""

    r = get_redis_conn()

    keys = r.keys("product*:sold")

    quantities_by_product = {}

    for key in keys:
        value = r.get(key)

        if value is None:
            continue

        product_id = int(key.split(":")[1])
        quantities_by_product[product_id] = int(value)

    best_sellers = sorted(
        quantities_by_product.items(),
        key=lambda x: x[1],
        reverse=True
    )

    return "".join(
        f"<li>Produit {pid}: {qty} vendu(s)</li>"
        for pid, qty in best_sellers[:10]
    )
```

Déploiement

(Le cas échéant, décrivez votre pipeline CI/CD et ce que vous avez appris dans ce laboratoire en ce qui concerne le déploiement. Il est obligatoire d'ajouter du code, des captures d'écran ou des sorties de terminal pour illustrer votre réponse.)

Pour ce déploiement, le CI déployé est plus défensif que le précédent étant donné qu'il adresse les problèmes potentiels de réutilisation de containers ou d'images Docker et élimine toutes données résiduelles Redis et MySQL des anciens runs. De plus, l

```
ERROR: for mysql: Cannot create container for service mysql: Conflict. The container name "/mysql" is already in use by container
"832aad4bfa47e99b4afdc101165d4269ad2277005769857dde6de94f71349124". You have to remove (or rename) that container to be able to reuse that name.
Encountered errors while bringing up the project.
Error: Process completed with exit code 1.
```

Ce laboratoire a toutefois permis de comprendre une différence entre les tests locaux et les tests sur serveurs : les tests sur serveurs ne peuvent pas utiliser DAO. En fait, ils doivent directement accéder aux données par requêtes MySQL. Par exemple, il faut deux implémentations différentes du rapport des plus gros acheteurs : un pour le scénario local et un autre pour le scénario serveur.

```
def get_highest_spending_users():
    """Get report of highest spending users"""

    r = get_redis_conn()

    keys = r.keys("order:*")

    expenses_by_user = defaultdict(float)

    #
    # Local : Redis/DAO
    #
    if keys:
        for key in keys:

            if ":item:" in key:
                continue

            data = r.hgetall(key)

            if not data:
                continue

            expenses_by_user[int(data["user_id"])] += float(
                data["total_amount"]
            )
```



```
#
# Server run MySQL
#
else:

    orders = get_orders_from_mysql()

    for order in orders:
        expenses_by_user[order.user_id] += float(
            order.total_amount
        )

highest_spending_users = sorted(
    expenses_by_user.items(),
    key=lambda item: item[1],
    reverse=True
)

result = []

for user_id, total in highest_spending_users[:10]:
    result.append(
        f"<li>{user_id}: {total:.2f}$</li>"
    )

return "".join(result)
```

```
# Cleanup leftover containers/volumes from previous runs
- name: Cleanup previous environment
  run: |
    docker compose down -v --remove-orphans || true
    docker rm -f mysql redis store_manager_cli || true

- name: Démarrer le conteneur
  run: |
    docker network create labo02-network || true
    docker compose up -d

- name: Attendre MySQL
  run: |
    sleep 10
    timeout=60
    until docker exec mysql mysqladmin ping -h localhost -u root -proot --silent; do
      echo "Waiting for MySQL..."
      sleep 3
      timeout=$((timeout - 3))
      if [ $timeout -le 0 ]; then
        echo "MySQL failed to start"
        exit 1
      fi
    done
    sleep 5
```

```
# Explicitly clear Redis
- name: Reset Redis
  run: |
    docker exec redis redis-cli FLUSHALL

# Explicitly clear MySQL
- name: Reset MySQL
  run: |
    docker exec mysql mysql \
      -uroot \
      -proot \
      -e "DROP DATABASE IF EXISTS labo02_db;"

    docker exec mysql mysql \
      -uroot \
      -proot \
      -e "CREATE DATABASE labo02_db;"
```

Le déploiement devait aussi palier le problème du des déploiements successifs échouant après le premier

```
/usr/bin/git config --global --add safe.directory /home/runner2/actions-runner/_work/log430-lab2/log430-lab2
Deleting the contents of '/home/runner2/actions-runner/_work/log430-lab2/log430-lab2'
Error: File was unable to be removed Error: EACCES: permission denied, unlink '/home/runner2/actions-runner/_work/log430-lab2/log430-lab2/src/__pycache__/config.cpython-311.pyc'
```

Ce laboratoire a aussi permis de voir la différence entre l'accès direct à la base de données avec MySQL comparée à l'utilisation d'un DAO comme intermédiaire.